

Tema 2

BASES DE DATOS NoSQL

ASIGNATURA: ARQUITECTURA DE DATOS (2026/2027)

DEPARTAMENTO DE INFORMÁTICA
UNIVERSIDAD CARLOS III DE MADRID

Objetivos de aprendizaje

- **Describir la arquitectura general** de una **plataforma de datos moderna** y los principios de las **BBDD NoSQL**.
- **Reconocer y diferenciar** los distintos **modelos de datos** (relacional, clave-valor, documentos, columnas y grafos), analizando sus características principales y aplicaciones prácticas.

Contenidos

- 2.1. Introducción a las bases de datos NoSQL
- 2.2. Modelos de datos. Modelos de agregación
- 2.3. Selección de una base de datos NoSQL

2.1. Introducción a las bases de datos NoSQL

- Contexto
- Normalización
- Escalabilidad
- Modelos de consistencia
- Ventajas e inconvenientes
- Comparación vs BBDD relacionales

Bases de datos NoSQL

Contexto

Orden antiguo

- Modelo **universal** basado en el principio de que la estructura de la información es **estática, fija** y el procesado es dinámico, variable.
- **Integración** funcional basada en el **modelo** de datos empresarial **único**.
- **Estandarización** de una arquitectura y de unos mecanismos de acceso (SQL).
- Concepto de **transacciones** para el control del acceso concurrente y para garantizar la **consistencia** de los datos.

Nuevo orden

- Orientado al **volumen** y a la **velocidad**.
- **No** siempre es absolutamente **imprescindible** la total **integridad** y **coherencia** de la información.
- **No** hay una solución **universal**, única y perfecta para todo.
- **Pragmatismo, flexibilidad, minimalismo**
- **BASE vs ACID**: podemos vivir con un sistema que se sabe que es incompleto e inexacto, bajo el supuesto de que “se pondrá al día” con la verdad.

No empieces por la herramienta. Empieza por el problema: qué datos, qué consultas, qué volumen y qué garantías necesitas.

Bases de datos NoSQL

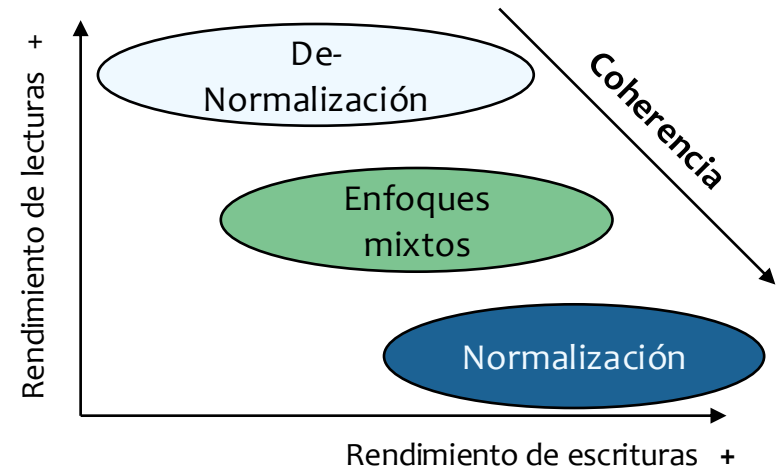
Normalización

- La normalización es un proceso muy utilizado en el modelo relacional.
- Su objetivo es reducir la redundancia y aumentar la coherencia de la información, proporcionando así una protección frente a posibles errores de actualización.
- Su mecanismo consiste en ubicar cada pieza de información en un único lugar al que acceden los elementos que la necesitan y lo hacen mediante referencias.
- Provoca una fragmentación y una red de vínculos que es necesario recorrer para volver a reconstruir la información tal como solemos utilizarla.
- Implica el uso masivo de operadores del tipo join.

Bases de datos NoSQL

Normalización

- El **mecanismo** utilizado para garantizar la **coherencia** tiene un **impacto en el rendimiento del sistema**.
- La normalización siempre es **deseable**. El **problema** está en lo que **cuesta conseguirla**.
- La reacción del mundo NoSQL es volver a poner el **foco en la aplicación** y los **casos de uso**.
- Se puede **aceptar cierto grado de incoherencia** dependiendo del tipo de la aplicación.



Si la consulta más frecuente exige muchos JOIN, ¿conviene almacenar el dato de otra forma para ese caso de uso?

Bases de datos NoSQL

Escalabilidad

- Tradicionalmente, las BBDD relacionales se han asociado más al escalado vertical, mientras que muchas BBDD NoSQL fueron diseñadas para facilitar el escalado horizontal.
- En las BBDD NoSQL, el procesamiento y almacenamiento de los datos se hace de manera distribuida a través de una red de nodos.
- La escalabilidad horizontal permite crecer en función de las necesidades del sistema distribuido, soportando añadir nuevos nodos a la red sin que afecte a su disponibilidad (adición en **caliente**).

ESCALABILIDAD
VERTICAL



ESCALABILIDAD
HORIZONTAL



- Scale up = máquina más potente.
- Scale out = más nodos trabajando conjuntamente.

Escalabilidad. Arquitectura distribuida

- En las **arquitecturas distribuidas**, la **fragmentación** se puede producir de forma horizontal y vertical.
 - **Fragmentación horizontal (sharding)**: división por **filas**, mediante **técnicas de dispersión** basadas en hash (BBDD Clave-valor) y en función del valor de atributos (BBDD orientadas a documentos, orientadas a columnas).
 - **Fragmentación vertical**: división por **columnas**, en función de estas (BBDD orientadas a columnas).
- Se caracterizan por la **replicación** masiva de los datos (en **diferentes servidores**).
- **Ventajas**:
 - Incrementar nivel de **paralelismo**.
 - Mejorar **eficiencia de consultas**.

Bases de datos NoSQL

Modelos de consistencia

- **Modelo relacional: ACID**
 - **Atomicidad:** Todo o nada.
 - **Consistencia:** Coherencia de los datos.
 - **Aislamiento:** Serialización de las transacciones.
 - **Durabilidad:** Los cambios son permanentes.
- **Modelo NoSQL: BASE**
 - **Disponibilidad básica** de los datos (*Basically Available*).
 - **Estado flexible** o transitorio de los datos (*Soft state*).
 - **Consistencia eventual** de los datos en el tiempo (*Eventual consistency*).

NoSQL **no significa** “sin transacciones” ni “sin consistencia”: **cada tecnología ofrece garantías concretas.**

Bases de datos NoSQL

Ventajas

- Enfoque hacia los **sistemas abiertos**.
- Orientación al **clúster masivo** y muy poco acoplado.
- Orientación hacia los **datos no estructurados**:
 - **Ausencia de esquema**.
 - **Relajación de la integridad y consistencia**.
 - **Flexibilidad** para adaptarse a nuevas situaciones.
- Son sistemas **políglotas**: permiten el uso **cada herramienta y cada producto** en el lugar en el que es **más efectivo**
- **No hay lenguaje estándar** para acceso:
 - No lenguaje SQL.
 - **Lenguaje propio** de manipulación de los datos, o acceso vía API.
 - Se proporcionan **drivers de acceso** a la BD para distintos lenguajes de programación.

Bases de datos NoSQL

Inconvenientes

- No soportan modelos de fiabilidad fuerte como ACID: son inadecuados para ciertos sistemas operacionales (transaccional bancario).
- Vuelta a trasladar complejidad hacia el código de las aplicaciones.
- No hay independencia lógica.
- No hay estándar: baja portabilidad.
- Complicado esquema de datos.
- La administración, seguridad y observabilidad dependen mucho de la tecnología y de su ecosistema.
- Madurez desigual según tecnología, proveedor y ecosistema.
- Curva de aprendizaje.

Bases de datos NoSQL

Comparación vs BBDD Relacionales

BBDD Relacionales	BBDD NoSQL
Modelo de datos único	Distintos tipos de modelos de datos
Esquema fijo	Esquema flexible
Datos estructurados	Datos estructurados, semi estructurados y no estructurados
Sin distribuir, distribuidas (modelos sencillos)	Altamente distribuidas
Propiedades ACID	Propiedades BASE

Bases de datos NoSQL

Conclusiones

- Se necesita una **BD NoSQL** si cumple con al menos una condición:
 - Es un entorno de aplicación que requiere **esquemas de datos flexibles** (en relación con los que ofrecen los relacionales).
 - Es un sistema **altamente distribuido** que necesita gestionar **grandes volúmenes de datos**, los cuales deben estar **siempre disponibles**.
- **Criterios de elección** del tipo de **BD NoSQL**:
 - Volumen de datos.
 - Concurrencia estimada.
 - N° de operaciones/unidad tiempo.
 - Tipos de operaciones más frecuentes.
 - Escalabilidad deseada.
 - Grado de integridad y consistencia deseado.
 - Naturaleza de los datos.

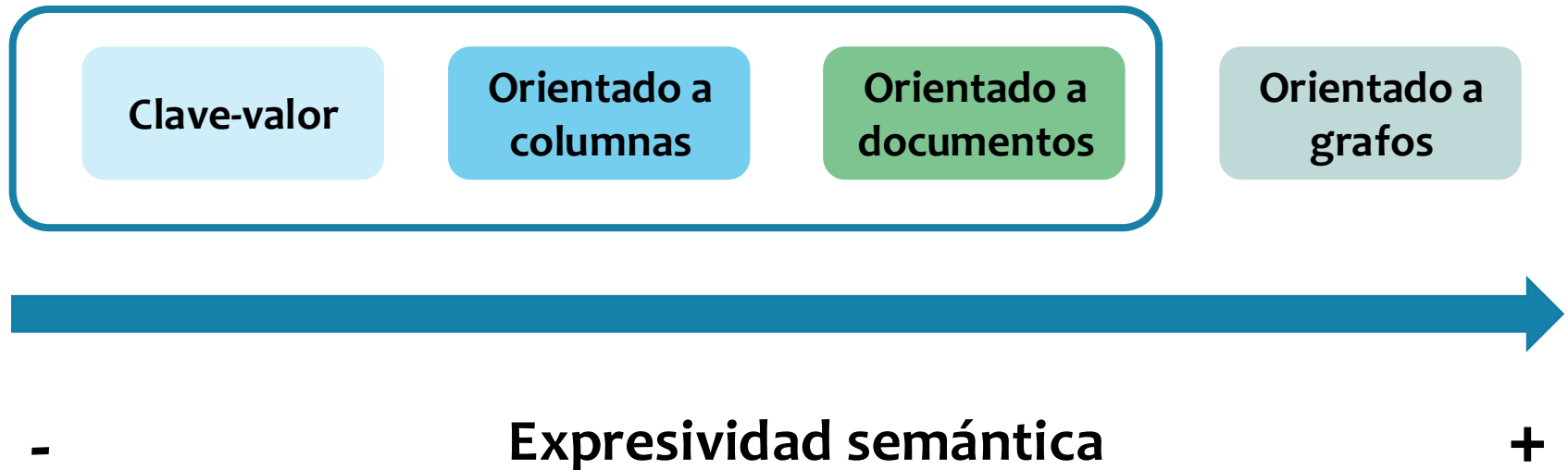
2.2. Modelos de datos

- Modelos de agregación
- Modelo clave-valor
- Modelo orientado a documentos
- Modelo orientado a columnas
- Modelo orientado a grafos

Modelos de datos

BBDD NoSQL

Modelos de agregación



Modelos de datos

Modelos de agregación

- Un **agregado** es un **conjunto de datos relacionados** que se gestionan como **una sola unidad**.
- **Características:**
 - Se identifican por una **clave única**.
 - **Unidad mínima de almacenamiento y consulta**.
 - Los **cambios** se aplican al **agregado completo**.
- Ejemplo gráfico:
 - **Relacional:** Pedido (tabla Pedidos) + Items (tabla DetallePedido).
 - **Agregado:** Documento JSON con pedido e ítems incrustados.

Modelos de datos

Modelos de agregación

- Agregado vs Relacional

Aspecto	Relacional (SQL)	Agregado (NoSQL)
Acceso	Varias tablas (JOINS)	Documento, familia de columnas,
Flexibilidad	Alta (consultas complejas)	Media (limitada a patrón de acceso)
Rendimiento	Penalización por JOINS	Optimizado para consultas típicas
Diseño	Normalización formal	Ad-hoc, guiado por el uso

Modelos de datos

Modelos de agregación

- **Diseño del agregado:**
 - Se guía por los **patrones de casos de uso** de la aplicación, que son las **consultas más frecuentes**.
 - El **perímetro del agregado** es una decisión de diseño: **define qué datos viajan siempre juntos**.
 - Bien diseñados, los agregados **reducen accesos** y **evitan joins costosos**.
 - El **proceso** es **menos formal** que en el modelo relacional y **más orientado a la práctica** (ad hoc).
 - Ejemplo:
 - En un e-commerce: guardar Pedido + Ítems del pedido → un único documento → una sola lectura.

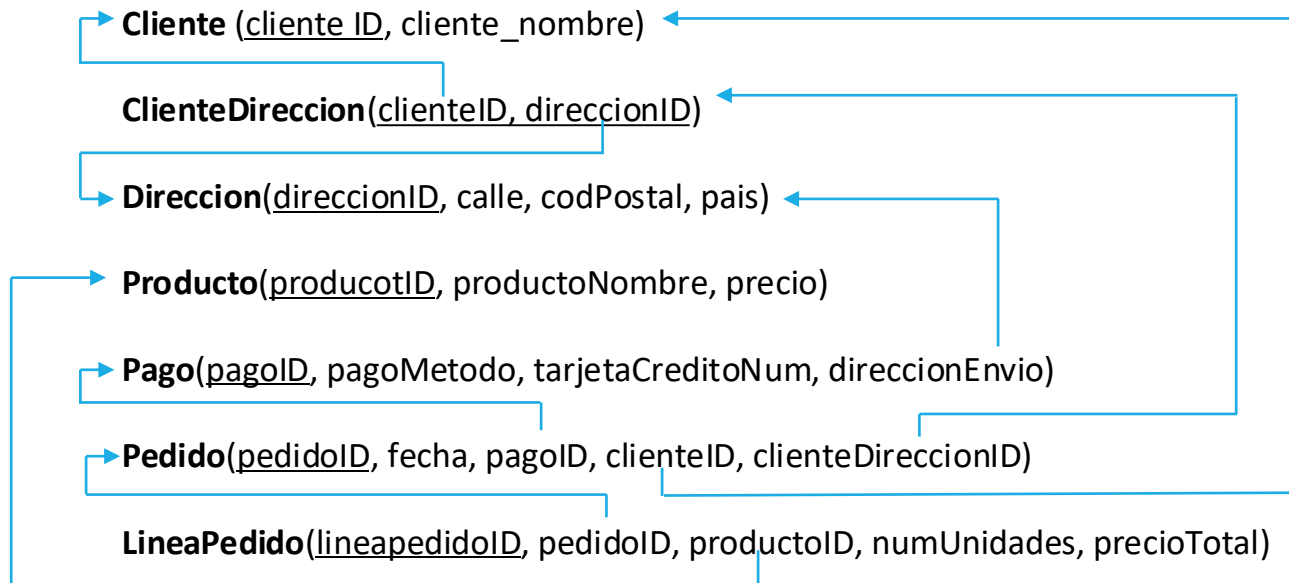
El agregado se diseña desde las consultas: **qué información debe viajar junta y qué coste aceptas al actualizarla.**

Modelos de datos

Modelos de agregación. Ejemplo

Ejemplo: Gestión de pedidos

- En **BD relacional** para recuperar un **Pedido** hay que hacer un **JOIN** de **Pedido**, **LineaPedido**, **Producto** y **Dirección**.

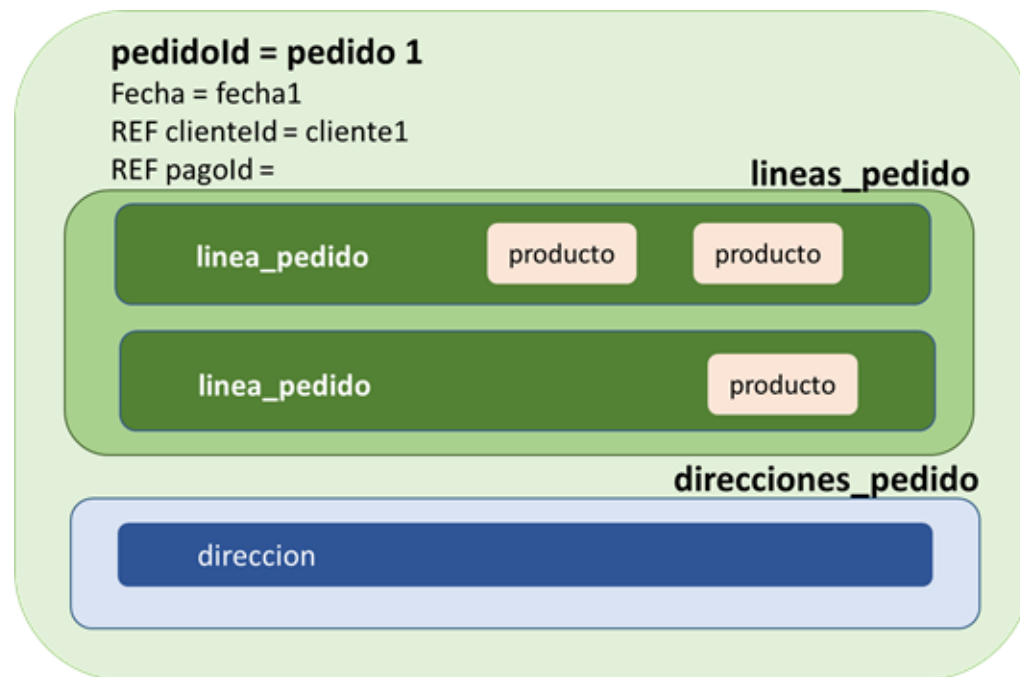


Modelos de datos

Modelos de agregación. Ejemplo

Ejemplo: Gestión de pedidos

- En BD NoSQL, el **Pedido** contiene toda la **información necesaria**:
líneas de pedido, productos y dirección.



Modelos de datos

Modelos de agregación

- **Esquema del agregado:**
 - En una **BD relacional** el esquema es **fijo**: todos los registros siguen la misma estructura.
 - En **NoSQL**, la **ausencia de esquema** otorga **flexibilidad** → permite almacenar **datos heterogéneos** (ej. cada documento JSON puede tener atributos distintos).
 - No es “ausencia total de esquema”: **el esquema se desplaza a la lógica de la aplicación**, lo que implica **riesgos** si no se controla.
 - Ejemplo: en MongoDB un documento Usuario puede tener {nombre, email} y otro además {dirección, teléfono}.

“**SCHEMALESS**” / Esquema flexible **no significa** datos sin diseño: parte del control pasa a la aplicación y a las prácticas de modelado.

Modelos de datos

Modelos de agregación

- Son útiles cuando:
 - La **funcionalidad está definida de antemano** y no se esperan cambios frecuentes.
 - **No existen relaciones complejas** entre entidades.
 - Los datos sufren **pocas actualizaciones**.
- **Escenarios típicos:**
 - Almacenamiento de **logs de usuario** (p.ej. clics en una web).
 - **Perfiles de usuario** en apps.
 - **Pedidos en e-commerce** (pedido + ítems en un solo documento).
 - **Gestión de contenidos** (blogs, prensa digital).
 - **Analítica web** en tiempo real.
 - **Series de datos de IoT** (ej. datos de sensores cada segundo).

Modelos de datos

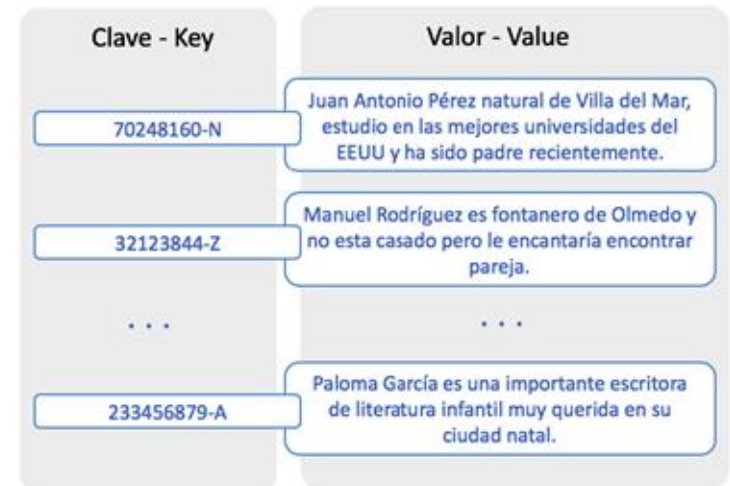
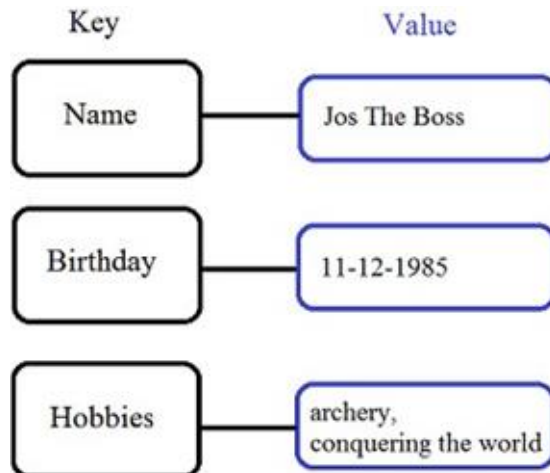
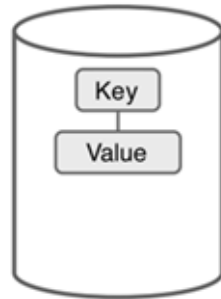
Modelos de agregación

- Tipos de modelos de agregación:
 - Clave-valor: estructura simple, solo permite búsquedas por clave.
 - Ejemplo: Redis (TEMA 3) para caché, sesiones y TTL.
 - Documentos: búsquedas por clave y por atributos internos (JSON/BSON).
 - Ejemplo: MongoDB (TEMA 4) para catálogos, perfiles y documentos JSON.
 - Columnas: datos organizados en familias de columnas para lecturas rápidas.
 - Ejemplo: Cassandra (TEMA 5) para eventos, históricos y grandes volúmenes distribuidos.

Modelos de datos

Modelos clave-valor

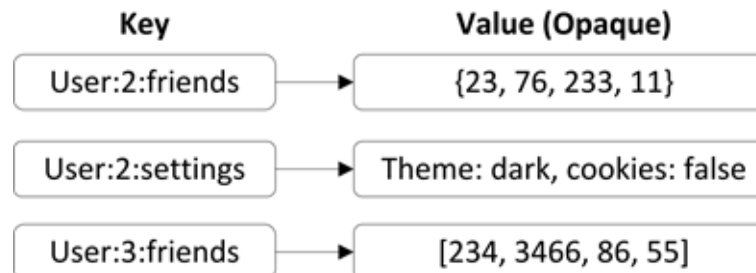
- El modelo de clave-valor es el **modelo más sencillo**.
- **Menor expresividad semántica.**
- Cada elemento se **identifica** de forma única por una **clave**:
 - **Clave → valor**
- La clave puede ser del dominio (DNI, NSS, e-mail, ...) o no.
- **Atomicidad** a nivel de clave.



Modelos de datos

Modelos clave-valor

- **Características:**
 - Alto **rendimiento** de **lectura/escritura**.
 - **Velocidad** en las **consultas**.
 - Fáciles de **escalar**.
 - Fáciles de **implementar**.
- La BD **no sabe de la estructura** del agregado (**caja negra**).
- El agregado como un **objeto opaco**.
- Si existe una **estructura** para el objeto, **será conocida** solamente por los **programas de aplicación** que acceden a la BD.



Modelos de datos

Modelos clave-valor

Ejemplo. Gestión de pedidos

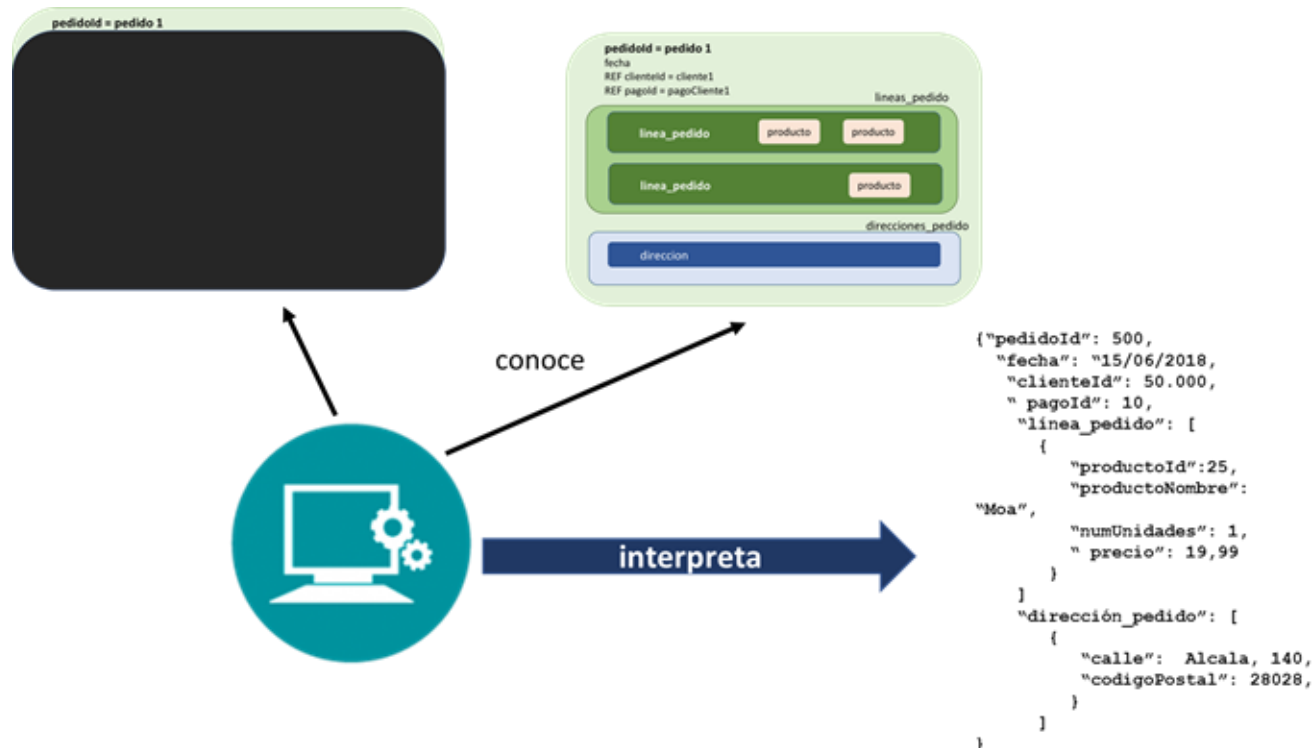


Modelos de datos

Modelos clave-valor

Ejemplo. Gestión de pedidos

- Es la **aplicación** la que da **estructura** al valor y lo **interpreta** (por ej., con un JSON, XML...).



Modelos de datos

Modelos clave-valor

- Los **SGBD** extienden los modelos de clave-valor.
- **Extienden la semántica** de los agregados e **incorporan relaciones** entre agregados.
- T3. Redis



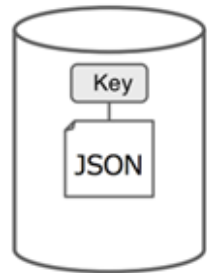
Necesitas recuperar un dato constantemente y ya conoces su clave.
¿Realmente necesitas hacer una consulta compleja?

=> si conozco la clave y necesito mucha velocidad => **clave-valor**

Modelos de datos

Modelos orientados a documentos

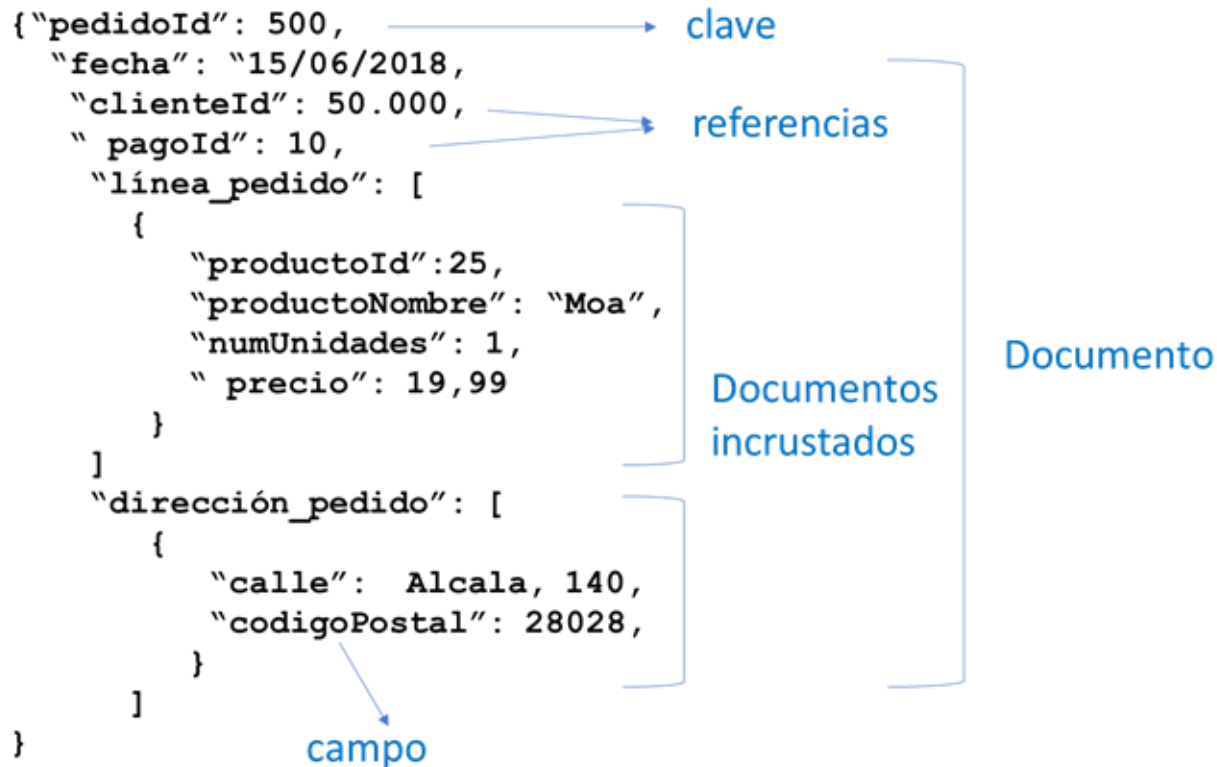
- **Extensión** del modelo **clave-valor**.
- Los **agregados** tienen una **estructura interna** que recibe el nombre de **documento** y que se almacena en formato **JSON**, **XML**... entre otros.
- La **BD** **conoce y sabe interpretar** esa **estructura interna**.
- A los agregados se puede **acceder**:
 - A través de la **clave**.
 - Al **contenido** a través de los **atributos del documento**.
- Los **documentos** se pueden agregar en **colecciones**.
- Sobre un documento, se puede **recuperar** todo su contenido, **modificar** una parte, **crear índices** sobre sus **atributos**...
- **Atomicidad** a nivel de **documento**.



Modelos de datos

Modelos orientados a documentos

Ejemplo: Gestión de pedidos



Modelos de datos

Modelos orientados a documentos

- Ejemplos de **SGBD** orientados a documentos
- T4. mongoDB

Un portátil tiene CPU y RAM; una camiseta, talla y color. ¿Tiene sentido obligar a todos los productos a tener exactamente la misma estructura?

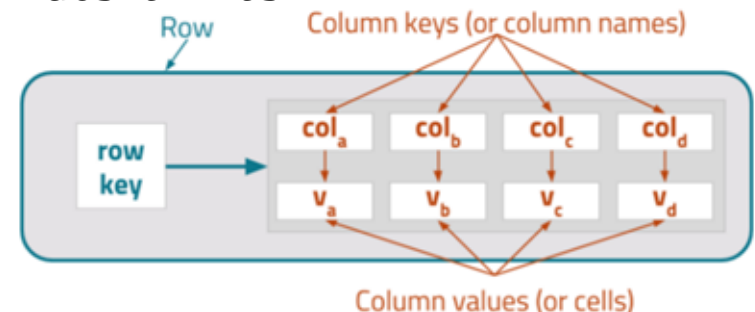
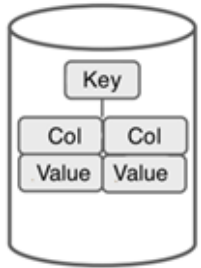
=> si los objetos tienen estructuras variables y se consultan como unidades => **documentos**.



Modelos de datos

Modelos orientados a columnas

- Ven los datos como una **matriz bidimensional**:
 - **Filas:** **agregaciones** de datos accedidas por **clave**.
 - **Columnas:** los atributos de las agregaciones se representan mediante la **tripleta**:
 - **<nombre, valor, timestamp>**
- **Atomicidad** a nivel de **fila**.
- Las **columnas** pueden agruparse en **familias** (una familia representa un concepto de la **agregación**).
 - Ejemplo: Agregado “Profesor” con dos familias:
 - Información personal.
 - Información académica.



Modelos de datos

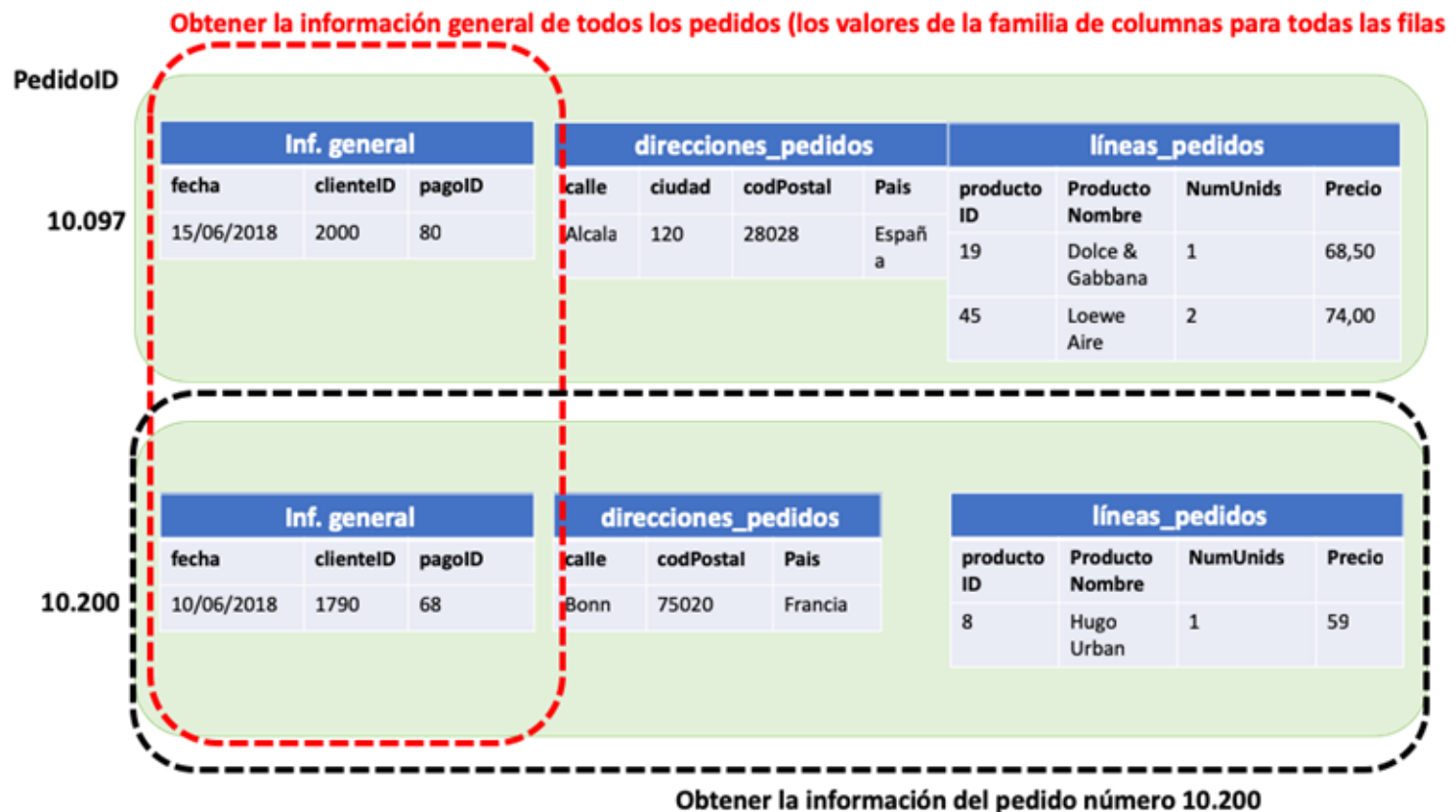
Modelos orientados a columnas

- Son adecuados para aplicaciones con **archivos distribuidos**.
- Son **menos escalables que las clave-valor**, son más lentas en **escritura**.
- **Más velocidad en lectura**.
- Son más **eficientes** cuando:
 - **Inserción de múltiples registros** al mismo tiempo (se actualizan bloques de columnas).
 - Cuando se quiere **acceder sólo a algunas columnas**.

Modelos de datos

Modelos orientados a columnas

Ejemplo: Gestión de pedidos



Modelos de datos

Modelos orientados a columnas

- Ejemplos de **SGBD** orientados a columnas
- T5 Cassandra



Millones de eventos llegan continuamente y ya se sabe qué consultas deberá soportar el sistema.

¿Diseñarías primero las entidades o las consultas?

=> en Cassandra, el modelo se diseña pensando en cómo se va a consultar.



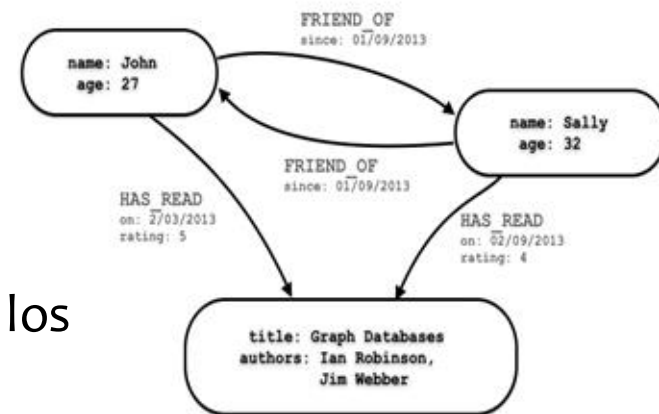
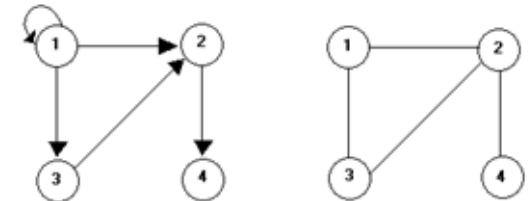
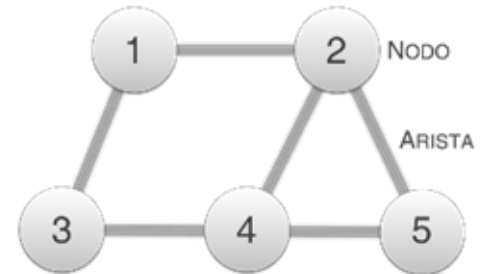
Google Bigtable



Modelos de datos

Modelos orientados a grafos

- El modelo en grafo utiliza **estructuras de grafo** para representar y almacenar los datos.
- Los grafos tienen dos **elementos básicos**:
 - **Nodos**: representan **conceptos y objetos** del mundo real.
 - **Aristas**: representan de forma explícita las **relaciones** entre nodos.
- **Tipos**:
 - Dirigidos y no dirigidos.
 - Etiquetados (se aporta semántica a los nodos y aristas) y de propiedad etiquetados (se asignan propiedades a los nodos y a las aristas).



Modelos de datos

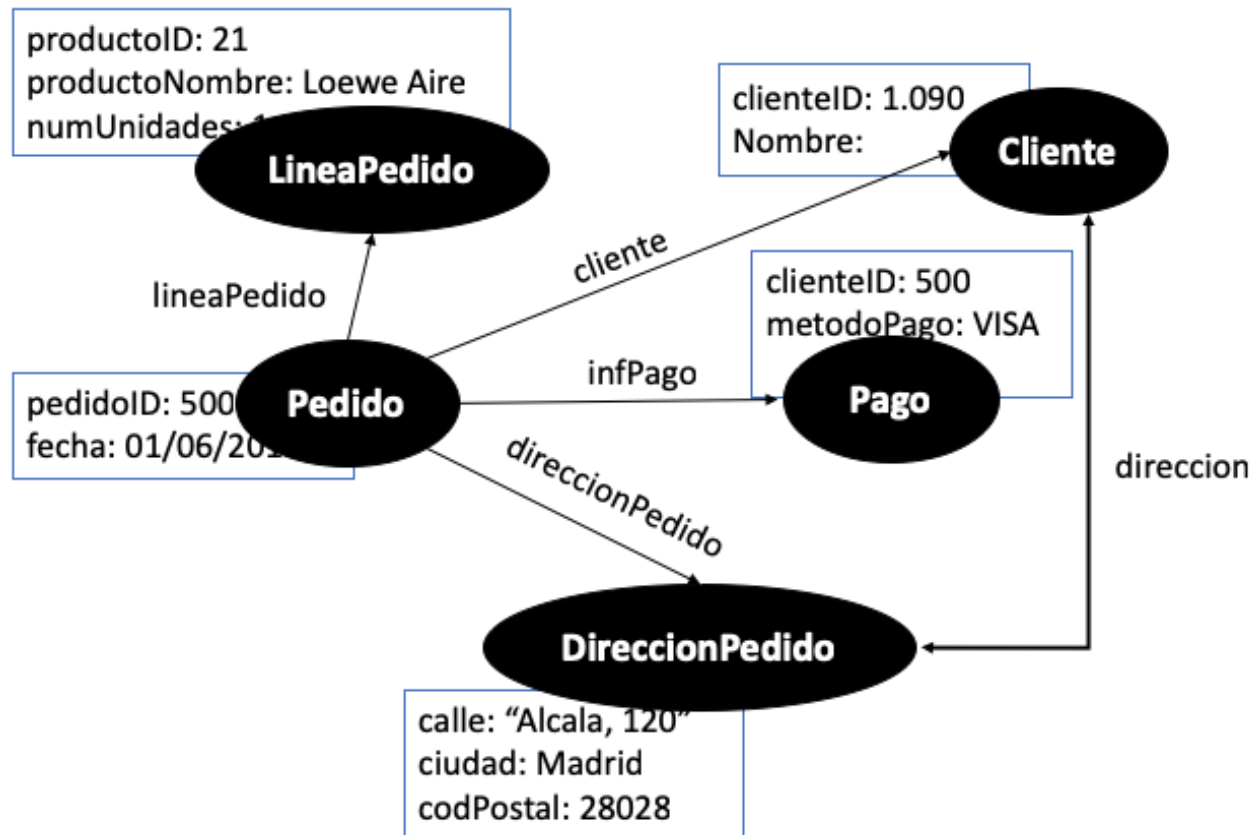
Modelos orientados a grafos

- **Datos altamente relacionados:** modelo útil cuando la **importancia de los datos** está en sus **interrelaciones** (pocos objetos y muchas relaciones).
- **Relaciones explícitas:** mejora en **tiempo de respuesta en las consultas** (navegación entre relaciones).
- **Difícilmente escalables.**
- El **esquema** está **implícito** en la estructura del grafo.
- Proporcionan **lenguajes de alto nivel**.
- Útiles cuando la **información** se puede representar como una **red**.
- **Ámbitos:** redes (RRSS, logística, mapas...), aplicaciones semánticas.

Modelos de datos

Modelos orientados a grafos

Ejemplo: Gestión de pedidos



Modelos de datos

Modelos orientados a grafos

- Ejemplos de **SGBD** orientados a grafos

¿Qué amigos de mis amigos han visto esta película?

¿Qué resulta más importante aquí: los datos de cada objeto o las relaciones entre ellos?

=> cuando el valor está en recorrer relaciones => **grafos**.



2.3. Selección de una base de datos NoSQL

- Criterios
- Ejemplos

Selección de una BD NoSQL

Criterios

- No existe una base de datos óptima para todos los problemas.
- La elección del modelo de datos depende de diferentes **criterios**:
 - **Requisitos** funcionales y no funcionales.
 - **Estructura** de los datos.
 - **Patrones** de lectura/escritura.
 - **Volumen y crecimiento**.
 - Necesidades de **consistencia**.
 - Necesidades de **disponibilidad**.
 - **Persistencia políglota**.

La tecnología es consecuencia de los requisitos, no el punto de partida.

Selección de una BD NoSQL

Criterios

Requisitos funcionales y no funcionales

- Los **funcionales** definen qué **operaciones del negocio** debe ejecutar la base de datos (como búsquedas complejas o transacciones).
- Los **no funcionales** determinan **atributos** de calidad como **latencia** o **rendimiento**.
- Evaluar ambos asegura que la tecnología elegida cumpla con las **reglas del sistema** sin comprometer los **niveles de servicio**.

Estructura de los datos

- La **naturaleza** de la información determina la BD NoSQL más adecuada.
- Elegir según la **variabilidad** o **jerarquía** del esquema evita almacenar la información en un formato ineficiente.

Selección de una BD NoSQL

Criterios

Patrones de lectura/escritura

- Analiza si la aplicación se caracteriza por una **carga** intensiva de **escrituras masivas**, **lecturas frecuentes** de baja latencia o un equilibrio entre ambas.
- Ayuda a seleccionar motores optimizados para el tipo de tráfico más frecuente.

Volumen y crecimiento

- Contempla el **tamaño actual** de los datos y la **velocidad** estimada a la que la base de datos **escalará en el tiempo**.
- Permite seleccionar modelos que soporten un **escalado horizontal (sharding)** fluido **sin degradar** el rendimiento a gran escala.

Selección de una BD NoSQL

Criterios

Necesidades de consistencia

- Establece si el sistema exige que todos los **nodos** devuelvan exactamente el **mismo dato al instante (consistencia fuerte)** o si tolera un **desfase temporal (consistencia eventual)**.
- Es un punto clave del **Teorema CAP**:
 - **Consistencia**: todos los clientes ven exactamente los mismos datos al mismo tiempo, sin importar a qué nodo se conecten.
 - **Disponibilidad**: Cada petición recibe una respuesta (éxito o fallo), garantizando que el sistema no se bloquee.
 - **Tolerancia a particiones**: el sistema sigue funcionando aunque la red falle y se rompa la comunicación entre nodos.

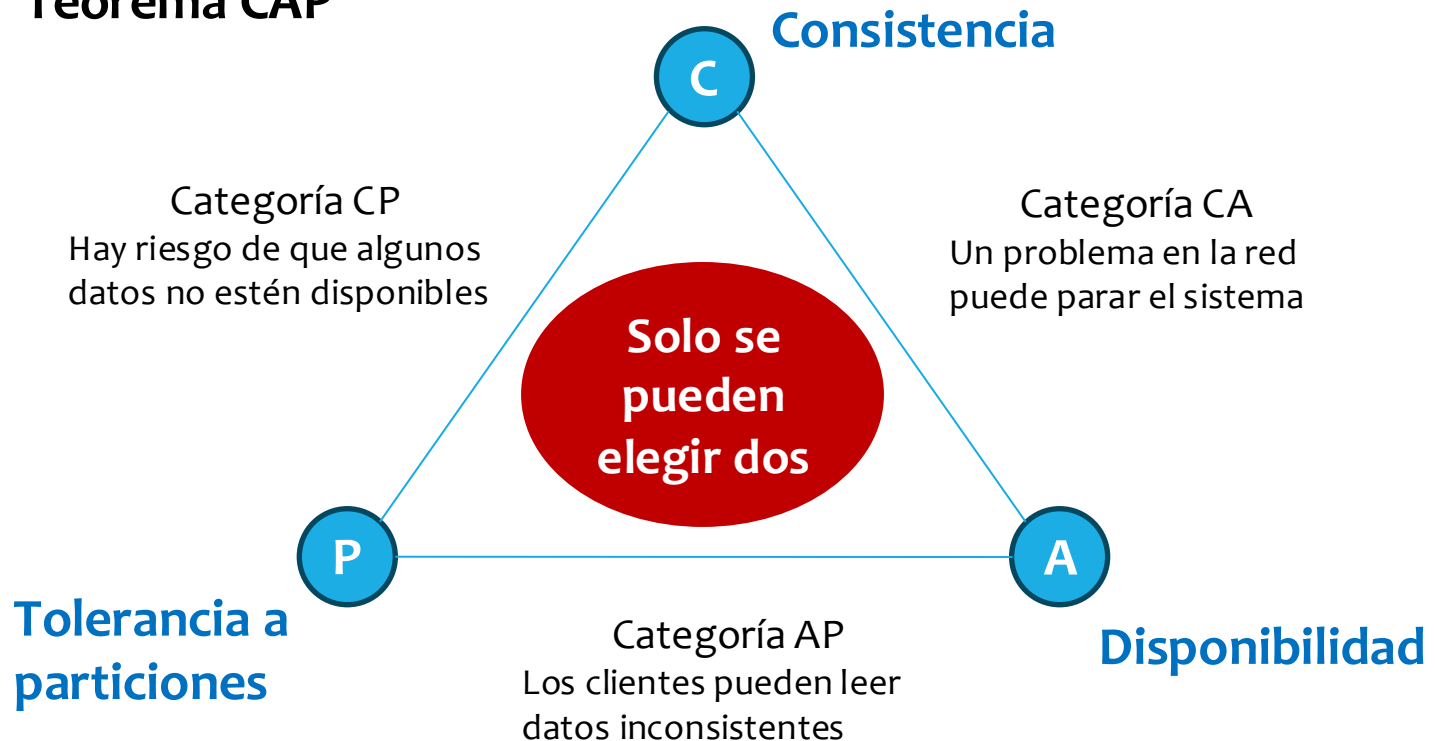
DECISIÓN DE DISEÑO: En sistemas distribuidos, la red puede fallar; a menudo la decisión real se sitúa entre la consistencia fuerte y la máxima disponibilidad.

Selección de una BD NoSQL

Criterios

Necesidades de consistencia (continuación)

- Teorema CAP



Selección de una BD NoSQL

Criterios

Necesidades de disponibilidad

- Mide el grado de **tolerancia a fallos** en los nodos para garantizar que el sistema siga operando sin interrupciones.
- En aplicaciones críticas se priorizarán motores distribuidos con la mayor capacidad de **replicación** y **redundancia**.

Persistencia políglota

- Contempla la combinación de **múltiples tecnologías** de almacenamiento dentro de **una misma arquitectura** para resolver **necesidades distintas** en cada módulo.
- Permite aprovechar la **fortaleza de cada motor** en lugar de depender de una única solución.

Selección de una BD NoSQL

Ejemplos

Ejemplo 1: Plataforma de streaming. Hacer una playlist completa.

- **Modelo Relacional**

- Plataforma de streaming (tipo Spotify/Netflix)
- Tablas: **Usuarios, Playlists, Canciones y Playlist_Canciones**
- Para reconstruir una playlist completa se necesita JOIN de 4 tablas

Tablas	Campos	Claves
Usuarios	id, nombre, email	
Playlists	id, usuario_id, nombre	FK → Usuarios.id
Canciones	id, título, artista, álbum	
Playlist_Canciones	playlist_id, cancion_id, orden	FK → Playlists, Canciones

Selección de una BD NoSQL

Ejemplos

Ejemplo 1: Plataforma de streaming. Hacer una playlist completa.

- **Diseño de agregado orientado a documento JSON (MongoDB)**

```
{
  "playlist_id": 321,
  "usuario": { "id": 88, "nombre": "Luis" },
  "nombre": "Favoritas verano",
  "canciones": [
    { "id": 1, "titulo": "Song A", "artista": "X" },
    { "id": 2, "titulo": "Song B", "artista": "Y" }
  ],
  "fecha_creacion": "2025-09-09"
}
```

Pros

- ✓ **Lectura rápida de playlist completa.**
- ✓ **Ideal para APIs y microservicios.**

Contras

- ✗ **Redundancia.** Duplicación de datos de canciones/artistas.
- ✗ **Actualizaciones masivas más costosas.**

Selección de una BD NoSQL

Ejemplos

Ejemplo 2: Geolocalización. Posiciones GPS del repartidor

■ Modelo Relacional

- Tablas:
 - **Repartidor** (id, nombre, telefono)
 - **EventoGPS** (id, repartidor_id, ts, lat, lon, precision_m, origen_app)
 - FK: EventoGPS.repartidor_id → Repartidor.id

```
SELECT ts, lat, lon
FROM EventoGPS
WHERE repartidor_id = 981
      AND ts BETWEEN '2025-09-09 10:00'
      AND '2025-09-09 11:00'
ORDER BY ts DESC;
```

Pros

- ✓ Integridad referencial.
- ✓ Fácil unir con otras tablas.

Contras

- ✗ Tablas enormes, índices grandes.
- ✗ Escalado más complejo.

Selección de una BD NoSQL

Ejemplos

Ejemplo 2: Geolocalización. Posiciones GPS del repartidor

- **Diseño orientado a columnas (Cassandra)**

```
CREATE TABLE gps_by_rider_day (  
  rider_id    bigint,  
  day         date,  
  ts          timestamp,  
  lat         double,  
  lon         double,  
  precision_m float,  
  origen_app  text,  
  PRIMARY KEY ((rider_id, day), ts)  
) WITH CLUSTERING ORDER BY (ts DESC);
```

Pros

- ✓ Muy eficiente para series temporales.
- ✓ Lecturas más rápidas por clave + rango.
- ✓ Escala horizontal fácil.

Contras

- ✗ Modelo query-first (casos de uso).
- ✗ Duplicación de datos.
- ✗ Controlar tamaño de particiones.

Selección de una BD NoSQL

Ejemplos

Ejemplo 2: Geolocalización. Posiciones GPS del repartidor

- **Diseño orientado a columnas (Cassandra)**
 - Ejemplo de partición en Cassandra

Partición: (rider_id=981, day='2025-09-09')

ts	lat	lon	precision_m	origen_app
2025-09-09 10:59:58	40.4201	-3.7049	6.2	android
2025-09-09 10:59:48	40.4200	-3.7051	7.1	android
2025-09-09 10:59:38	40.4197	-3.7053	5.8	android
...				

- El “agregado” es la partición optimizada para la consulta objetivo (rider+día), con eventos ordenados por tiempo.

Conclusiones

¿SQL o NoSQL?

- La pregunta no es “¿SQL o NoSQL?”, sino “¿qué necesita este caso de uso?”
 - ¿Qué datos tengo y cómo cambian?
 - ¿Cómo se consultan y cómo se escriben?
 - ¿Qué volumen, latencia y concurrencia debo soportar?
 - ¿Qué consistencia, disponibilidad y tolerancia a fallos necesito?
 - ¿Hace falta una única base de datos o una arquitectura polígloa?

No hay una respuesta universal: hay una decisión razonada para un contexto concreto.

Conclusiones

Ejemplo: ¿qué tecnología elegirías?

- Plataforma de streaming. ¿Qué modelo encaja mejor en cada necesidad?
 - Sesiones de usuario, caché y TTL → clave-valor / Redis
 - Catálogo con atributos variables → documentos / MongoDB
 - Histórico masivo de eventos → wide-column / Cassandra
 - Pagos y facturación → relacional / SQL
- Relaciones complejas entre usuarios, actores, títulos y géneros → grafos

Una plataforma tipo Netflix (T7) combina varias tecnologías: sesiones/caché, catálogo, eventos, pagos y analítica no tienen los mismos requisitos.

Otros tipos de BD

Series temporales

Optimizadas para datos ordenados en el tiempo: métricas, sensores, monitorización.

Ej.: InfluxDB, TimescaleDB

Búsqueda

Índices, texto completo, relevancia y recuperación rápida de información.

Ej.: Elasticsearch, OpenSearch

Vectoriales

Similitud entre vectores/embeddings, búsqueda semántica y aplicaciones de IA.

Ej.: Milvus, Weaviate

Multimodelo

Un mismo sistema soporta varios modelos o formas de acceso.

Ej.: ArangoDB, Cosmos DB

Referencias

- Eric Redmond. Seven Databases in Seven Weeks: A Guide to Modern Databases and the NoSQL Movement. 2012
- Harrison et al. Next Generation Databases: NoSQL, NewSQL, and Big Data. 2015. Apress.
- Kristina Chodorow. MongoDB: The Definitive Guide. 2013. O'Reilly Media, Inc.
- Ian Robinson, Jim Webber, and Emil Eifré. Graph Databases 2nd Edition. 2015. O'Reilly Media
- McCreary, D., & Kelly, A. (2013). Making sense of NoSQL: A guide for managers and the rest of us. Manning Publications.
- Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly.

Bibliografía Adicional

artículos

- Rick Cattell. 2011. **Scalable SQL and NoSQL data stores**. SIGMOD Rec. 39, 4 (December 2010), 12–27. <https://doi.org/10.1145/1978915.1978919>
- Seth Gilbert and Nancy Lynch. 2002. **Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services**. SIGACT News 33, 2 (June 2002), 51–59. <https://doi.org/10.1145/564585.564601>
- Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. 2007. **Dynamo: amazon's highly available key-value store**. SIGOPS Oper. Syst. Rev. 41, 6 (December 2007), 205–220. <https://doi.org/10.1145/1323293.1294281>
- Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. 2008. **Bigtable: A Distributed Storage System for Structured Data**. ACM Trans. Comput. Syst. 26, 2, Article 4 (June 2008), 26 pages. <https://doi.org/10.1145/1365815.1365816>
- Avinash Lakshman and Prashant Malik. 2010. **Cassandra: a decentralized structured storage system**. SIGOPS Oper. Syst. Rev. 44, 2 (April 2010), 35–40. <https://doi.org/10.1145/1773912.1773922>
- Michael Stonebraker, Samuel Madden, Daniel J. Abadi, Stavros Harizopoulos, Nabil Hachem, and Pat Helland. 2018. The end of an architectural era: it's time for a complete rewrite. Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker. Association for Computing Machinery and Morgan & Claypool, 463–489. <https://doi.org/10.1145/3226595.3226637>